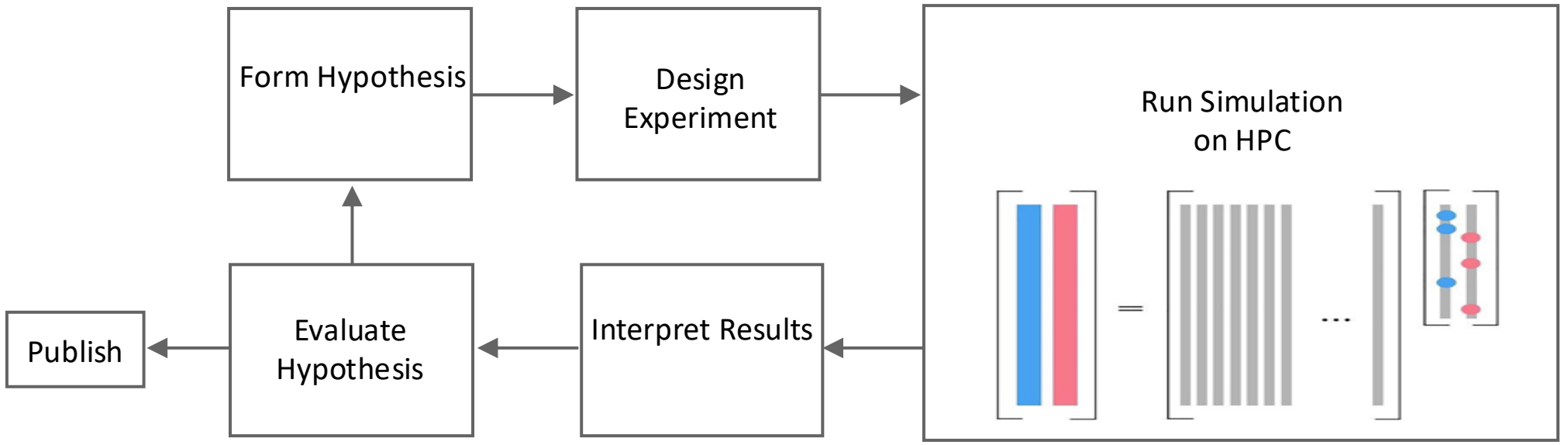
# AI Agents for Scientific Workflows

Luke Smith
Research Associate
Scalable Computational Intelligence
March 24, 2026

# Scientific Workflows Defined

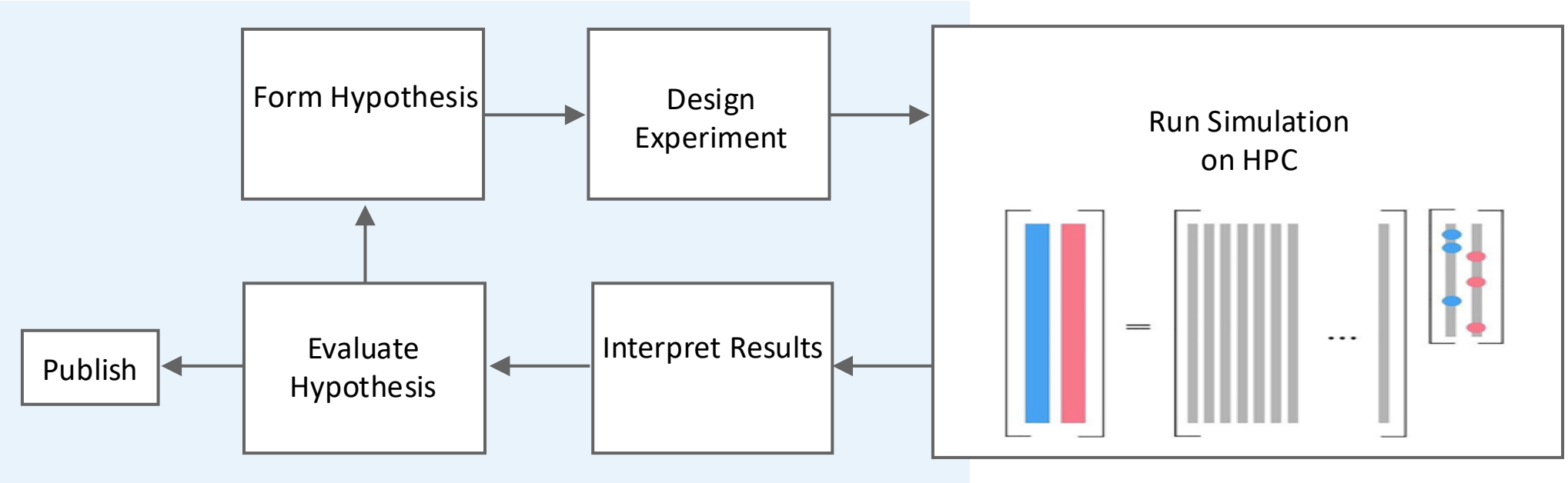
A scientific workflow can be seen as any process that incorporates computation into the **scientific method**. Computation can be numerical simulation, data processing, or visualization.




A scientific workflow is typically represented as a **Directed Acyclic Graph (DAG),** in which nodes represent tools/actions and edges represent the flow of data.

# Scientific Workflows Defined

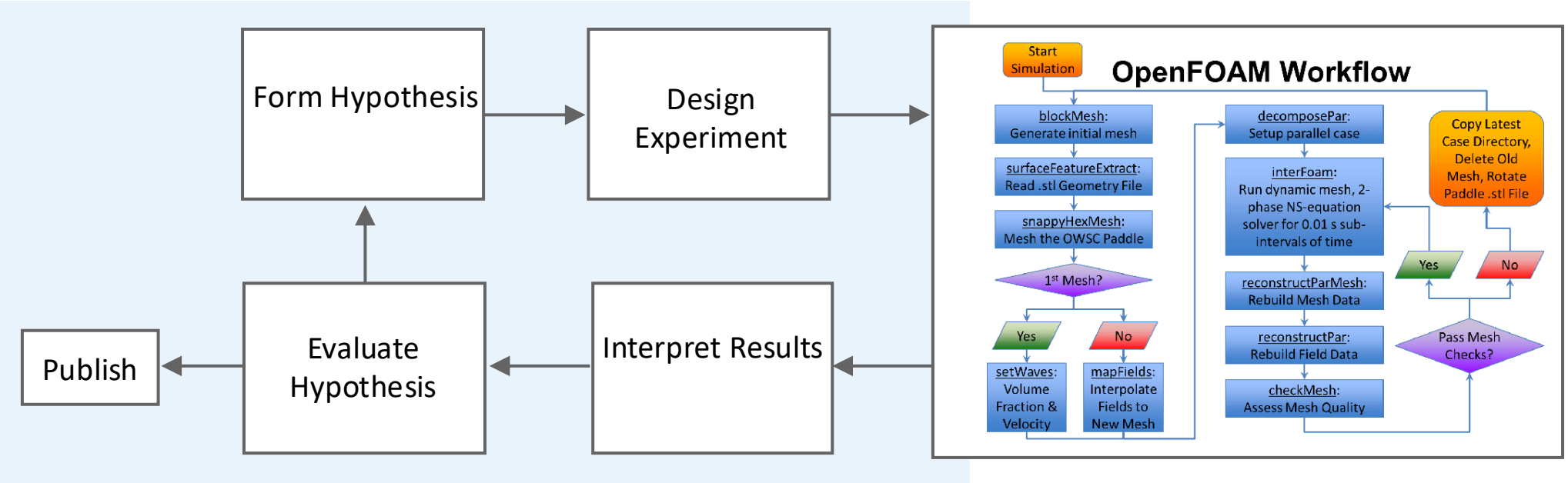
A scientific workflow can be seen as any process that incorporates computation into the **scientific method**. Computation can be numerical simulation, data processing, or visualization.




The bulk of scientific reasoning occurs on the left-hand side of this diagram, which represents the process of refining a hypothesis based on observation.

# Scientific Workflows Defined

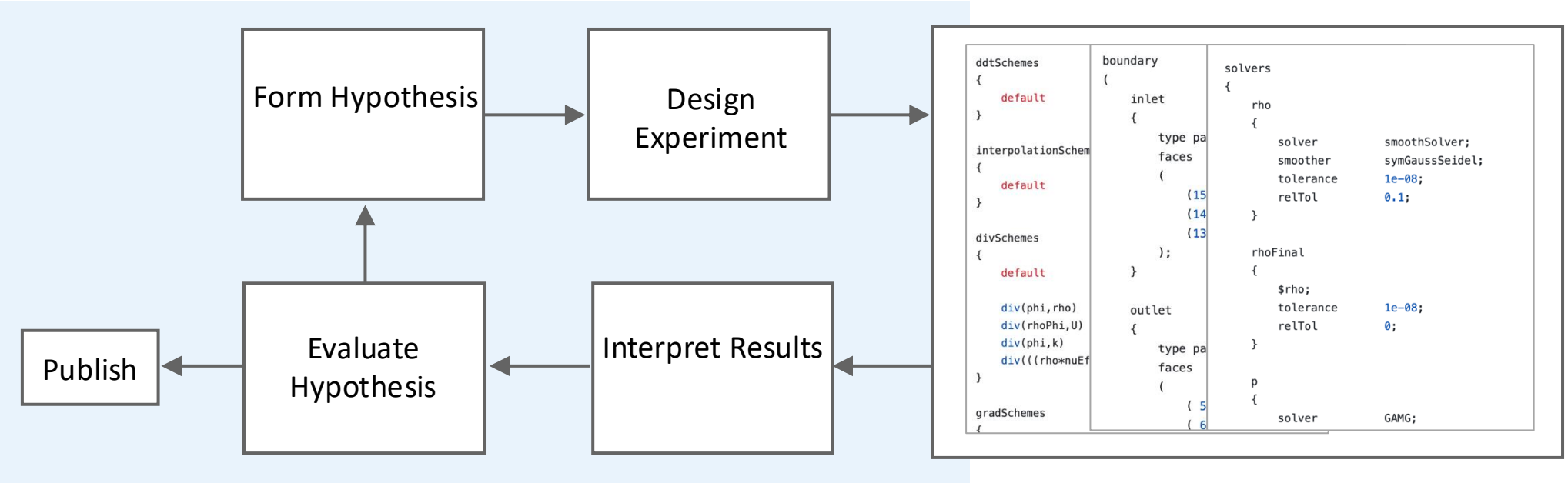
A scientific workflow can be seen as any process that incorporates computation into the **scientific method**. Computation can be numerical simulation, data processing, or visualization.




However, with growing software stacks and complex legacy code bases, computational scientist often spend undue time on the right-hand side of this diagram. It can require several years for a new student to become familiar with just one or two of these workflows.

# Scientific Workflows Defined

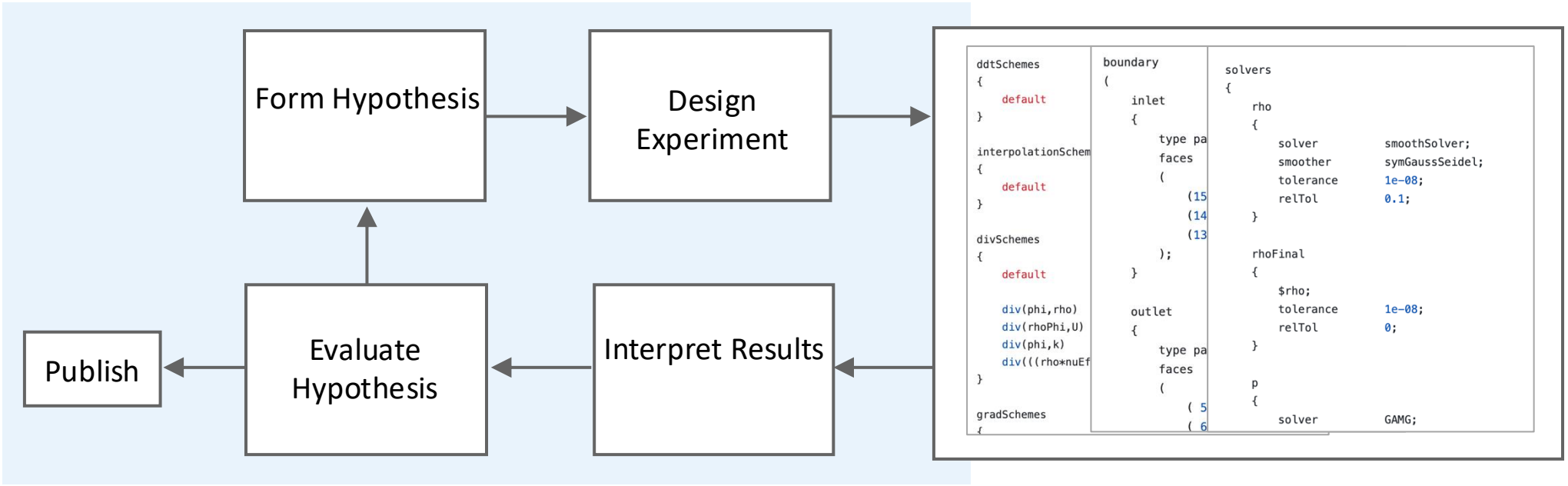
A scientific workflow can be seen as any process that incorporates computation into the **scientific method**. Computation can be numerical simulation, data processing, or visualization.




However, with growing software stacks and complex legacy code bases, computational scientist often spend undue time on the right-hand side of this diagram. It can require several years for a new student to become familiar with just one or two of these workflows.

# Scientific Workflows Defined

A scientific workflow can be seen as any process that incorporates computation into the **scientific method**.
Computation can be numerical simulation, data processing, or visualization.

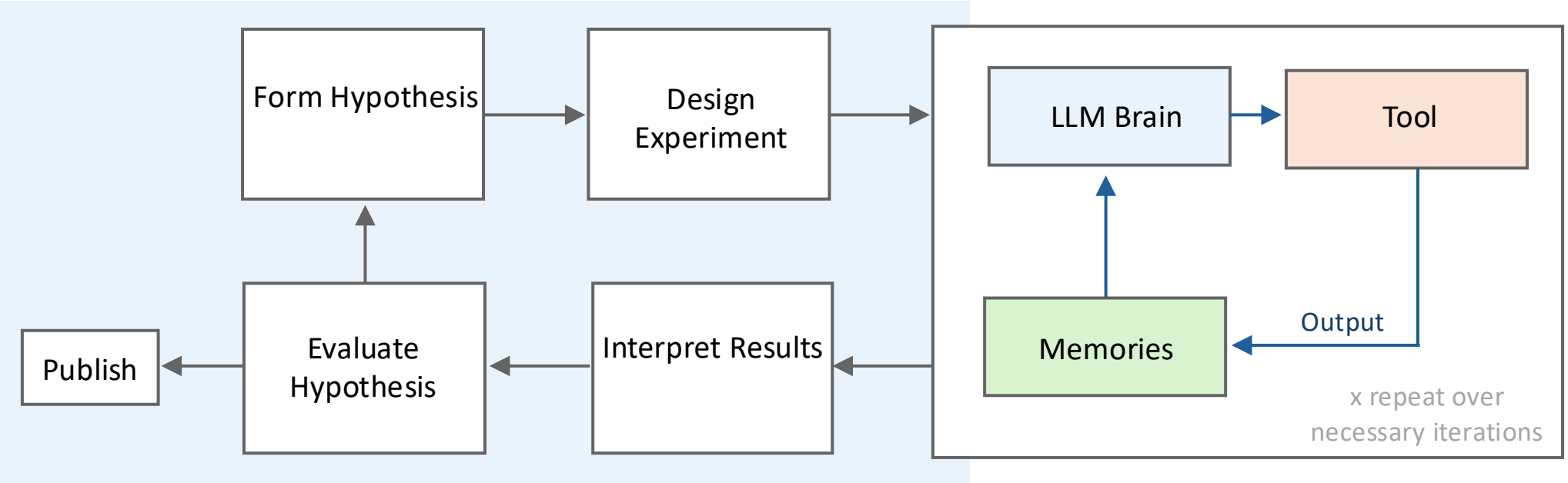



**Goal:**

Shift time investment in scientific workflows from cumbersome software implementation back to scientific reasoning and interpretation.

# How can AI agents help?

AI agents enable us to automate the setup and execution of a simulation or experiment, allowing the user to focus refining their hypothesis and designing new experiments.

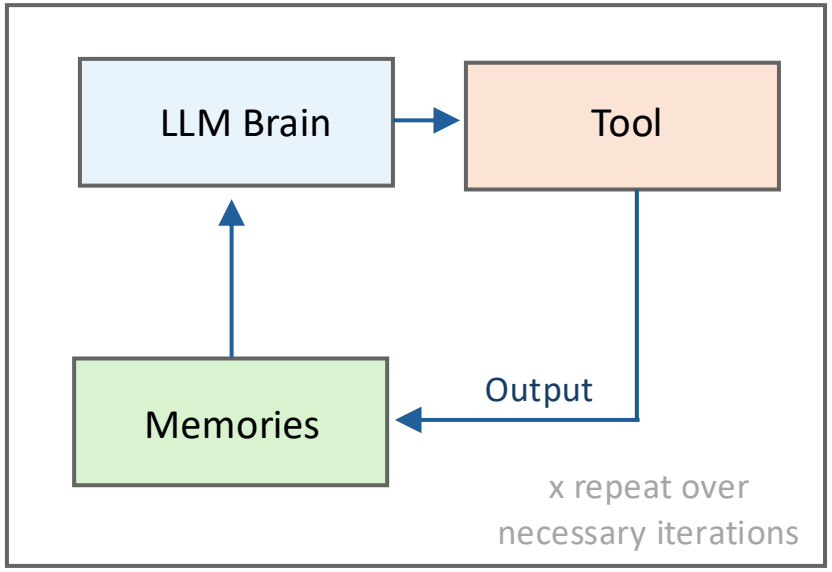


# How can AI agents help?

AI agents enable us to automate the setup and execution of a simulation or experiment, allowing the user to focus refining their hypothesis and designing new experiments.

In the context of scientific workflows:

- **LLM Brain:**
  Backend neural network with reasoning module (multistep planning or ReAct)
- **Tool:**
  Physics simulator, data processing scheme
- **Memories:**
  Agent prompt, previous observations, domain-specific context (user manuals, style-guides, etc.)

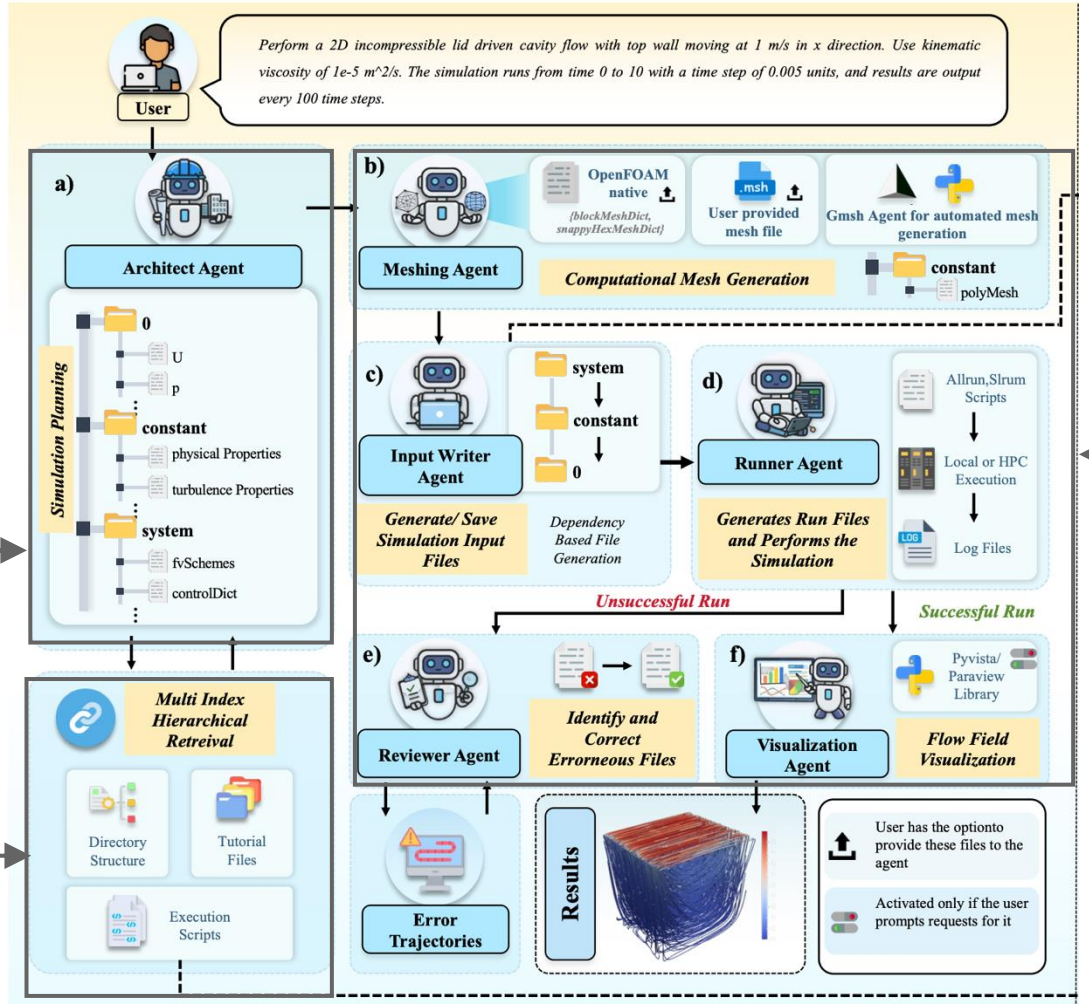



We automate a given scientific workflow by mapping to the three core components of an agentic system: reasoning, tools, and memories.

# Example 1: Foam-Agent for Computational Fluid Dynamics

**LLM Brain:**
Claude with orchestration schema a dynamic planning.

**Memories:**
Observations appended with tutorials, sample scripts, and manual via RAG.

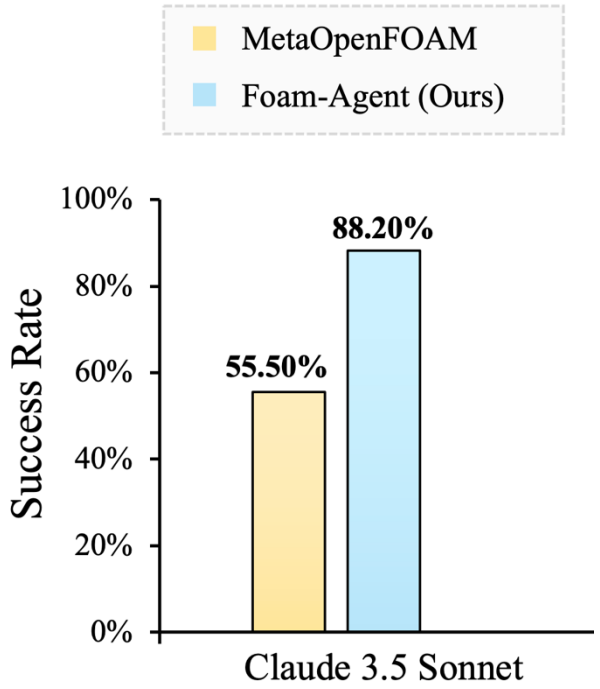


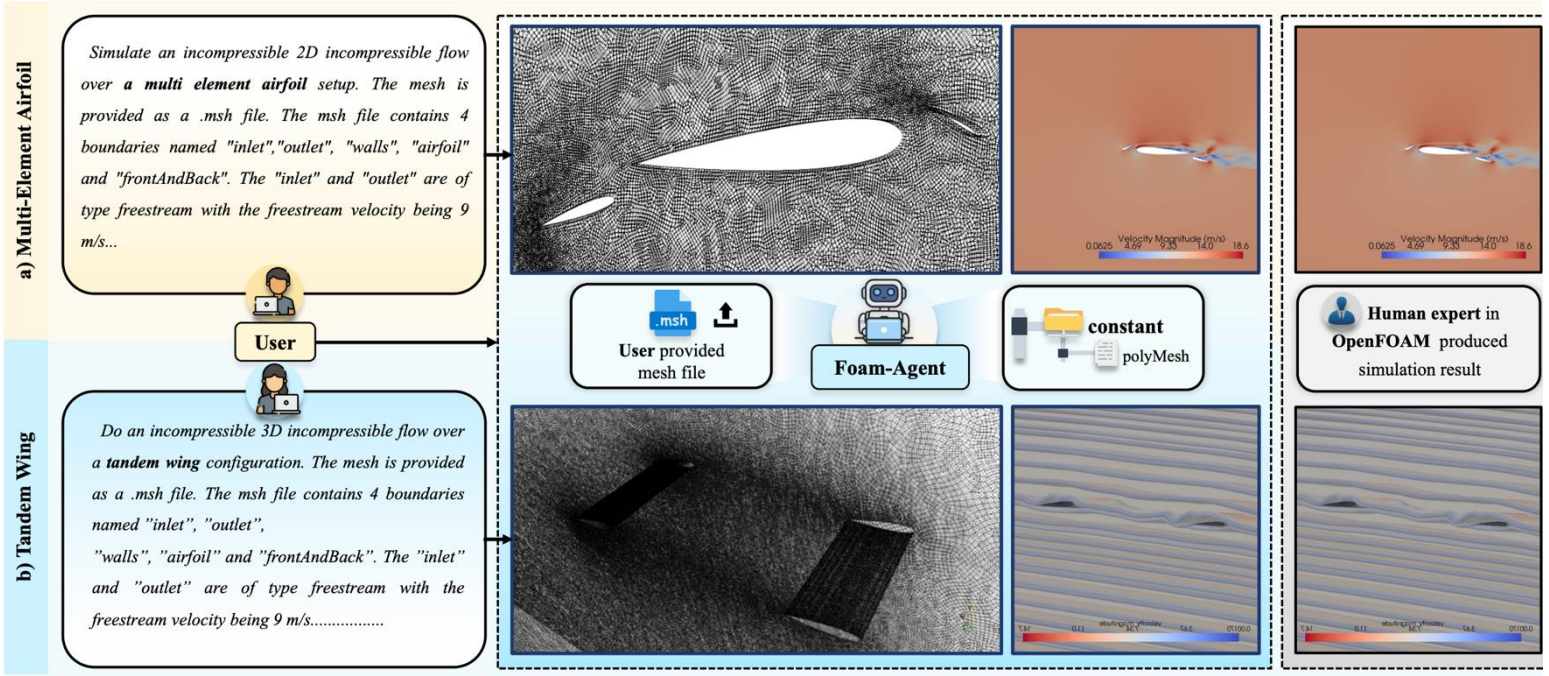

**Tools:**
Meshing
Input File Writing
Running Solver
Reviewing Outputs
Visualization

Each tool is implemented by a separate Pydantic schema and prompt.

Source: Yu et al, "Foam Agent: Toward Automated Intelligent CFD Workflows" (2026)

# Example 1: Foam-Agent for Computational Fluid Dynamics

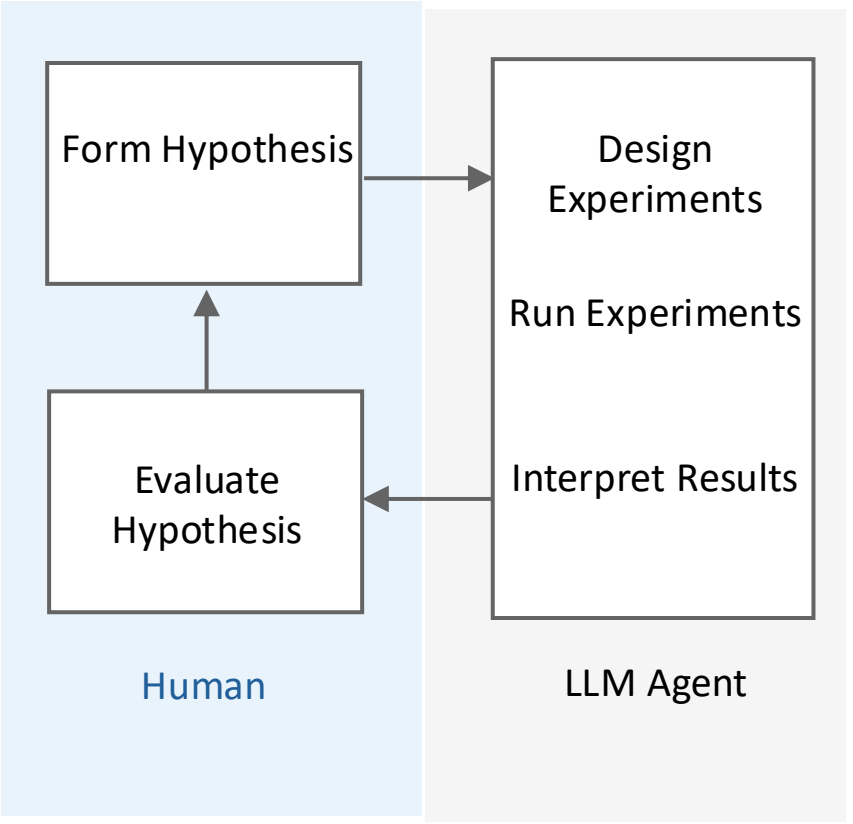



Source: Yu et al, "Foam Agent: Toward Automated Intelligent CFD Workflows" (2026)

- The author's report a success rate of about 90% on a benchmark of canonical steady flows.
- Success was defined based on the presence of a stable output flowfield. Benchmark metrics based on the **fidelity** of the flowfield were not reported.
- Still, those who have run OpenFOAM know that arriving at a stable solve is one of the more difficult steps of the workflow. Fidelity is typically achieved by fine-tuning a base configuration like this.

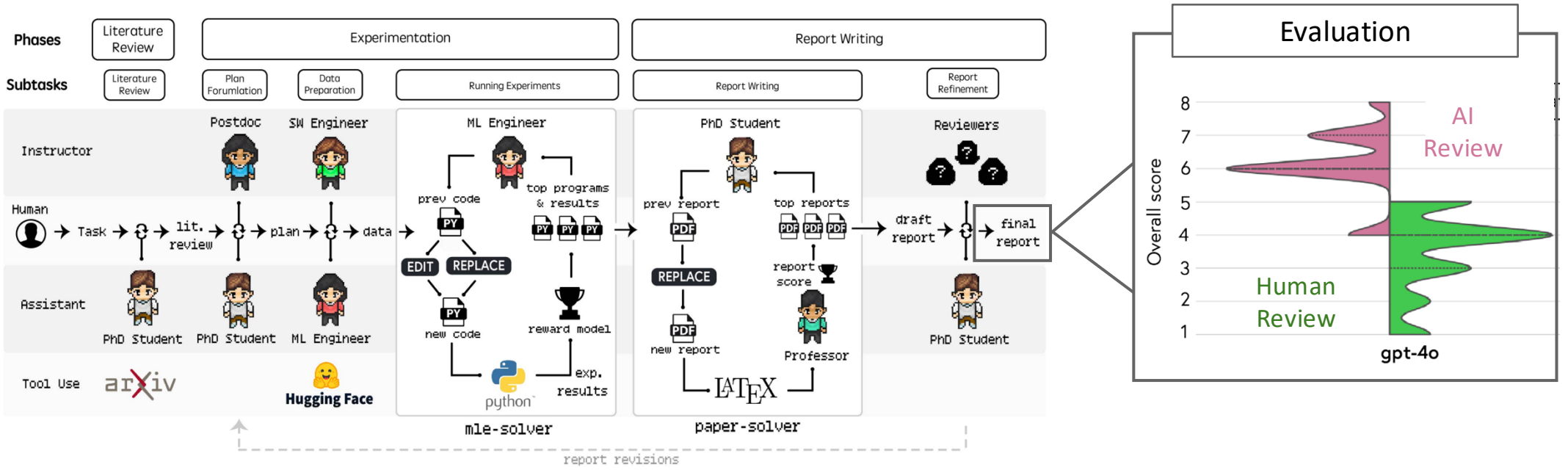
# Example 2: Self-Driving Labs


Source: Vertical Cloud Lab at BYU (https://github.com/vertical-cloud-lab)

- More aggressive workflows permit the agent to make decisions about which experiments to run – ideal when one expects a large parameter space with a small subset of interesting experiments.
- Note that agent automation is not limited to numerical simulations! Any system with a software interface can be designated as an agent's tool call.

# Example 3: Agent Laboratory

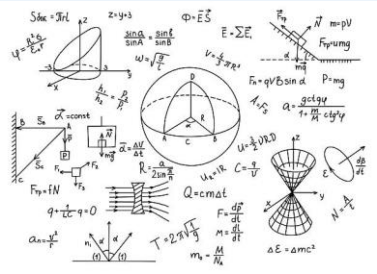



Source: Schmidgall et al, "Agent Laboratory: Using LLM Agents as Research Assistants" (2025)

- More extreme workflows attempt to automate the **entire end-to-end research process**, including formulating a hypothesis, performing a literature review, running experiments, and writing a paper.
- The problem with these workflows is analogous to error accumulation: the more agents we incorporate, the more bias we ultimately introduce. In this case, that manifests in poor-quality papers.

# Roadmap

## Domain Expertise

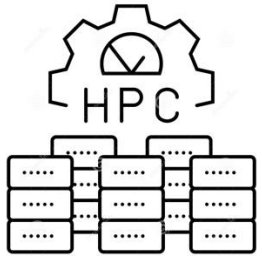


**Problem**: LLM's are generalist models, and scientific computing applications are highly specialized.

**Potential Solutions**: Custom schema for each piece of software (MPC), advanced reasoning and prompting, RAG, higher-capacity backends.

This session

## HPC Guardrails

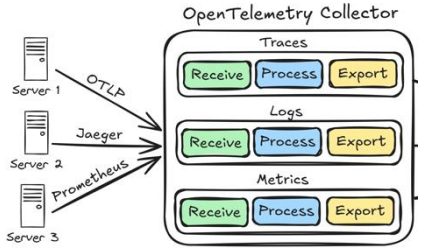



**Problem**: In an HPC context, agents must adhere to compute, memory, and security, constraints.

**Potential Solutions**: Custom schema for HPC applications, containerization, incorporating an evaluation loop into the agent's reasoning.

After lunch: Amit

## Evaluation




**Problem**: LLM backends provide no means of monitoring or verifying their activity.

**Potential Solutions**: Functional scorers, LLM as judge, implement tracing as a simultaneous process (Arize Phoenix).

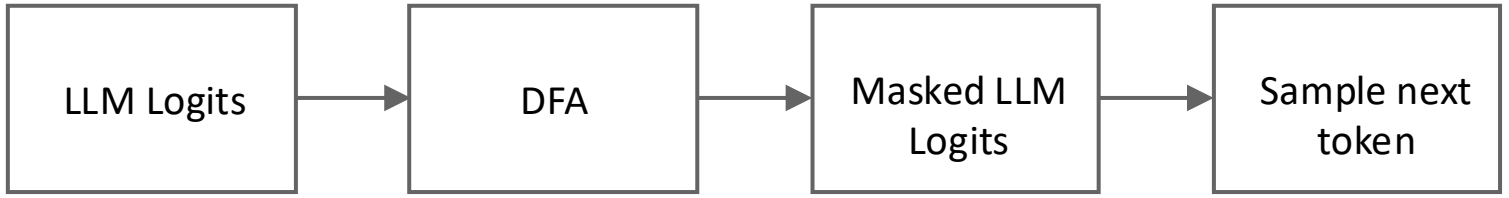
After lunch: Juliana

# Demonstration

Our goal is to set up a tool call for an LLM to generate code. In most agent setups, tool calls are implemented via schema for structured generation. **Pydantic schema** look something like this:

```
class custom_schema(BaseModel):
        thought: str = Field(…,description="What to do next")
        property1: Optional[str] = Field(…, description = "")
        property2: Optional[str] = Field(…, description = "")
```

The idea is that the LLM's logit's are constrained such that it returns a response that adheres to the specified schema. The enforcement of schema can be implemented with the **outlines** library, which masks invalid logits based on a pre-computed map for each state in the schema:




**Deterministic Finite Automaton:**

Input: current state within schema
Output: list of valid tokens

# Demonstration

In the demonstration that follows, we implement a GenerateCode schema. We constrain the LLM such that it returns either code or a final summary. The agent's reasoning loop is then based on whether it generated code for the user's request: